

Informe Técnico Experiencia 3

Proyecto PCer2

Sistema de gestión para servicio técnico de computadores

Dato	Información
Asignatura	FullStack I
Evaluación	Experiencia 3
Equipo	Jael Yapur, Felipe Moya, Sarai Jara
Fecha	19/06/2026
Repositorio	https://github.com/fmoyam/pcer2-yapur-jara-moya-010D

Índice

1. **Ficha del documento**
2. **Introducción general del proyecto**
3. **a. Contenido del informe entregado en la Experiencia 1**
 - i. Propósito del sistema
 - ii. Ámbito del sistema
 - iii. Descripción general
 - iv. Requisitos funcionales
 - v. Requisitos no funcionales
 - vi. Historias de usuario
 - vii. Product Backlog priorizado
 - viii. Estimación de esfuerzo de las tareas
4. **b. Diagramas actualizados**
 - i. Diagrama de arquitectura actualizado
 - ii. Modelo de base de datos actualizado
 - iii. Puertos y bases de datos por microservicio
5. **c. Servicio de autenticación**
 - i. ¿En qué consiste la autenticación por token?
 - ii. ¿Cómo se implementa la autenticación por token?
 - iii. Microservicio service-auth
 - iv. API Gateway y AuthenticationFilter
 - v. Flujo de autenticación en PCer2
 - vi. Evidencias de autenticación
6. **d. Implementación por capas**
 - i. Explicación general de la arquitectura por capas
 - ii. Capa Controller
 - iii. Capa DTO
 - iv. Capa Service
 - v. Capa Repository
 - vi. Capa Model / Entity
 - vii. Ejemplo de implementación por capas usando service-software

- viii. Evidencias de código
- ix. Evidencias en Postman: POST, GET, GET(id), PUT(id), DELETE(id)
- x. Comunicación entre microservicios
- xi. Ejemplo de comunicación mediante WebClient en service-orden

7. e. Swagger / OpenAPI

- i. ¿Qué es Swagger?
- ii. ¿En qué consiste Swagger?
- iii. ¿Cómo se implementa Swagger en microservicios Spring Boot?
- iv. Dependencias utilizadas
- v. Anotaciones utilizadas: @Tag, @Operation y @Schema
- vi. OpenApiConfig y token Bearer en Swagger
- vii. Ejemplo con evidencias en 3 microservicios
- viii. Swagger centralizado en API Gateway

8. f. Testing

- i. ¿En qué consiste el testing?
- ii. ¿Por qué es importante testear?
- iii. Pruebas unitarias
- iv. Patrón AAA: Arrange, Act, Assert
- v. Testing con JUnit 5 y Mockito
- vi. Testing implementado por microservicio
- vii. Documentación del 60% del testing implementado
- viii. Evidencias de métodos testeados
- ix. Evidencias de resultados BUILD SUCCESS

9. Conclusión

Ficha del documento

Fecha	Revisión	Autor	Modificación
18/06/2026	01	Equipo PCer2	Actualización del informe para Experiencia 3
19/06/2026	02	Equipo PCer2	Reestructuración según pauta: puntos a-f, autenticación, Swagger y testing

Nombre integrante	Rol / responsabilidad
Jael Yapur	Desarrollo de service-servicio, service-orden, service-software, service-mantenimiento y apoyo en api-gateway.
Felipe Moya	Desarrollo de service-clientes, service-equipo, service-estadísticas, service-hardware y apoyo en api-gateway.
Sarai Jara	Desarrollo de service-voucher, service-auth, service-descuento y apoyo en api-gateway.

Introducción general del proyecto

El presente informe tiene como finalidad documentar la evolución técnica del proyecto **PCer2**, correspondiente a un sistema de gestión para un servicio técnico de computadores.

En la primera experiencia se definieron los requisitos iniciales del sistema, sus usuarios, alcance, funciones principales, historias de usuario y planificación general. En la Experiencia 3, el proyecto evoluciona hacia una solución backend basada en microservicios, incorporando nuevos servicios, autenticación por token, documentación Swagger/OpenAPI, pruebas unitarias con JUnit 5 y Mockito, comunicación entre microservicios y API Gateway.

PCer2 permite administrar información relacionada con clientes, equipos, servicios técnicos, órdenes de trabajo, vouchers, hardware, software, productos de mantenimiento, descuentos, autenticación y estadísticas internas.

a. Contenido del informe entregado en la Experiencia 1

i. Propósito del sistema

El propósito de PCer2 es entregar una solución backend para administrar la información de un servicio técnico de computadores, permitiendo registrar clientes, equipos, servicios técnicos, órdenes de trabajo, vouchers, hardware, software, productos de mantenimiento, descuentos, autenticación y estadísticas internas.

El propósito de PCer2 es entregar una solución backend para administrar la información de un servicio técnico de computadores, permitiendo registrar clientes, equipos, servicios técnicos, órdenes de trabajo, vouchers, hardware, software, productos de mantenimiento, descuentos, autenticación y estadísticas internas.

ii. Ámbito del sistema

- Registrar clientes y equipos asociados.
- Administrar servicios técnicos y órdenes de trabajo.
- Generar vouchers y aplicar descuentos.
- Registrar hardware, licencias de software y productos de mantenimiento.

- Consultar estadísticas internas.
- Autenticar usuarios mediante token JWT.
- Centralizar rutas mediante API Gateway.
- Documentar APIs mediante Swagger/OpenAPI.
- Validar lógica de negocio mediante testing unitario.

El sistema no considera pagos en línea ni facturación electrónica. Su uso está orientado al personal interno del servicio técnico.

iii. Descripción general

PCer2 se desarrolló con Java y Spring Boot bajo una arquitectura de microservicios. Cada microservicio cumple una responsabilidad específica, posee su propio puerto de ejecución y utiliza su propia base de datos. El API Gateway funciona como punto de entrada centralizado y permite aplicar autenticación antes de acceder a los microservicios protegidos.

iv. Requisitos funcionales

Código	Requisito funcional
RF-01	El sistema debe permitir registrar clientes.
RF-02	El sistema debe permitir registrar equipos asociados a clientes.
RF-03	El sistema debe permitir registrar servicios técnicos.
RF-04	El sistema debe permitir generar órdenes de trabajo.
RF-05	El sistema debe permitir generar vouchers.
RF-06	El sistema debe permitir registrar hardware.
RF-07	El sistema debe permitir registrar licencias de software.
RF-08	El sistema debe permitir registrar productos de mantenimiento.
RF-09	El sistema debe permitir administrar descuentos.
RF-10	El sistema debe permitir consultar estadísticas internas.
RF-11	El sistema debe permitir autenticación mediante token.
RF-12	El sistema debe permitir documentar endpoints mediante Swagger/OpenAPI.
RF-13	El sistema debe permitir consumir microservicios mediante API Gateway.
RF-14	El sistema debe permitir probar endpoints mediante Postman.
RF-15	El sistema debe permitir validar la lógica mediante pruebas unitarias.

v. Requisitos no funcionales

Código	Requisito no funcional
RNF-01	El sistema debe estar desarrollado en Java con Spring Boot.
RNF-02	El sistema debe utilizar arquitectura de microservicios.
RNF-03	Cada microservicio debe tener responsabilidad propia.
RNF-04	Los endpoints deben estar documentados con Swagger/OpenAPI.

Código	Requisito no funcional
RNF-05	El sistema debe contar con pruebas unitarias usando JUnit 5 y Mockito.
RNF-06	El repositorio debe utilizar control de versiones mediante GitHub.
RNF-07	El proyecto debe contar con README.md formateado en Markdown.
RNF-08	El sistema debe incluir archivo SQL con poblamiento de base de datos.
RNF-09	El sistema debe poder ejecutarse y probarse en ambiente local.
RNF-10	El acceso debe considerar autenticación mediante token.

vi. Historias de usuario

Código	Historia de usuario
HU-01	Como técnico, quiero registrar un cliente con sus datos para tener su información disponible en futuras atenciones.
HU-02	Como técnico, quiero asociar uno o más equipos a un cliente para llevar un historial de sus dispositivos.
HU-03	Como técnico, quiero registrar servicios realizados con su precio para generar el voucher de cobro.
HU-04	Como administrador, quiero ver estadísticas de clientes frecuentes para apoyar la gestión interna.
HU-05	Como técnico, quiero buscar un cliente por RUT o nombre para acceder rápidamente a su historial.
HU-06	Como técnico, quiero registrar una orden de trabajo para controlar el servicio solicitado por el cliente.
HU-07	Como técnico, quiero registrar software asociado a una atención para mantener trazabilidad de licencias utilizadas.
HU-08	Como técnico, quiero registrar productos de mantenimiento para controlar insumos usados.
HU-09	Como administrador, quiero autenticar usuarios mediante token para proteger el acceso al sistema.
HU-10	Como administrador, quiero gestionar descuentos para aplicarlos en servicios o vouchers.

vii. Product Backlog priorizado

Prioridad	Historia	Esfuerzo	Dependencias
1	HU-01 Registrar cliente	5	Ninguna
2	HU-02 Asociar equipos a cliente	8	HU-01
3	HU-06 Registrar orden de trabajo	8	HU-01, HU-02
4	HU-03 Registrar servicios y voucher	13	HU-01, HU-02, HU-06
5	HU-07 Registrar software	5	HU-06
6	HU-08 Registrar productos de mantenimiento	5	HU-06
7	HU-04 Ver estadísticas	8	HU-01, HU-02, HU-06
8	HU-09 Autenticación por token	8	Usuarios del sistema
9	HU-10 Gestión de descuentos	5	HU-03, HU-06

viii. Estimación de esfuerzo de las tareas

Tarea	Historia asociada	Horas estimadas
Diseño de base de datos	Todas	8
Implementar CRUD de clientes	HU-01	6
Implementar CRUD de equipos	HU-02	8
Implementar CRUD de servicios	HU-03	6
Implementar órdenes de trabajo	HU-06	10
Implementar voucher	HU-03	8
Implementar estadísticas	HU-04	6
Implementar hardware	HU-02	6

Tarea	Historia asociada	Horas estimadas
Implementar software	HU-07	6
Implementar mantenimiento	HU-08	6
Implementar autenticación	HU-09	8
Implementar descuentos	HU-10	6
Configurar API Gateway	Todas	8
Implementar Swagger	Todas	8
Implementar testing	Todas	10
Preparar informe y evidencias	Todas	8

b. Diagramas actualizados

i. Diagrama de arquitectura actualizado

El sistema PCer2 utiliza una arquitectura de microservicios. El usuario, Postman o un posible frontend consumen los endpoints a través del API Gateway. Este gateway centraliza las rutas, aplica autenticación mediante token y redirige hacia el microservicio correspondiente.

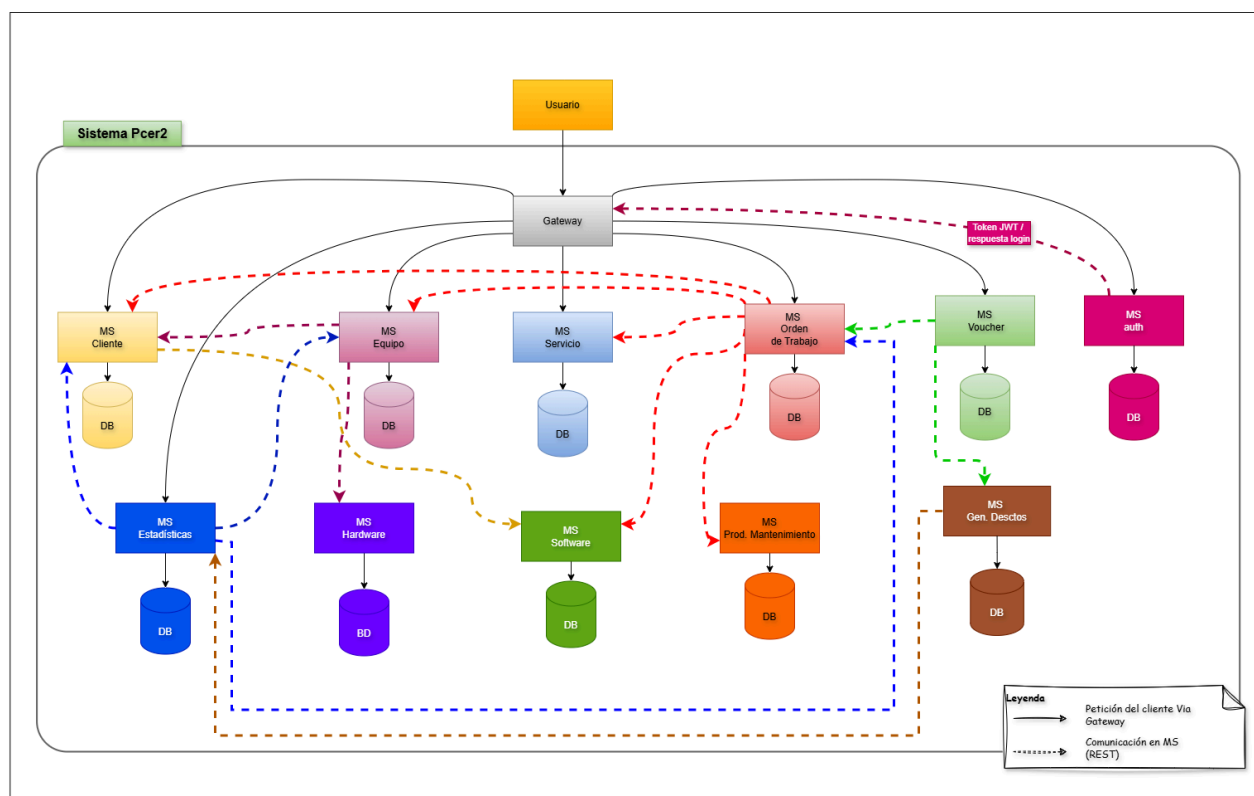
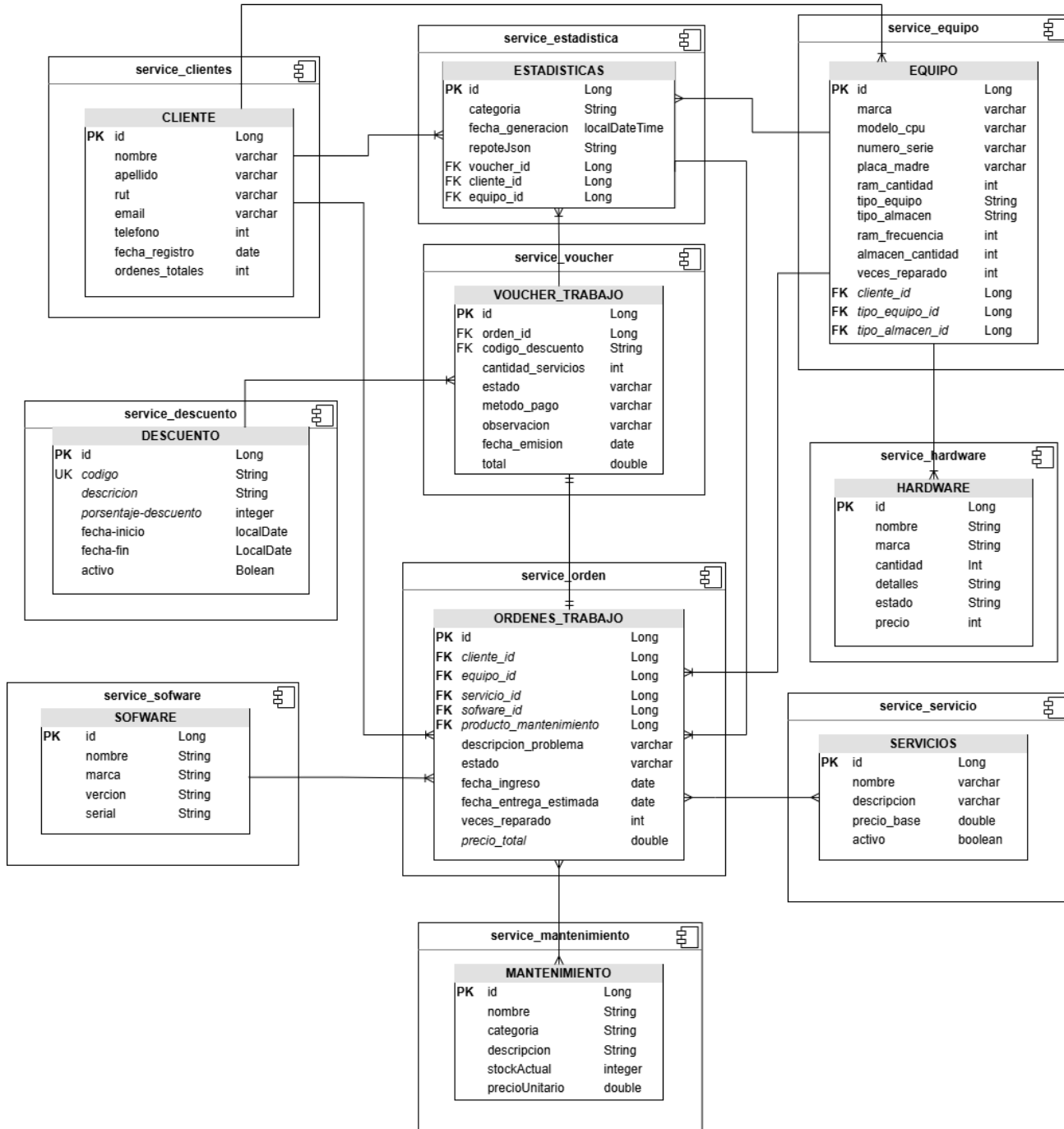


Figura 1. Diagrama de arquitectura actualizado del sistema PCer2.

ii. Modelo de base de datos actualizado

Cada microservicio posee su propia base de datos, manteniendo independencia entre módulos. Esta organización permite evitar dependencia directa entre tablas de distintos servicios y facilita el mantenimiento del sistema.



iii. Puertos y bases de datos por microservicio

Microservicio	Responsable	Puerto	Base de datos	Endpoint principal
api-gateway	Equipo	8080	No aplica	/
service-clientes	Felipe Moya	8081	pc_clientes	/api/v1/clientes
service-equipo	Felipe Moya	8082	pc_equipos	/api/v1/equipos
service-servicio	Jael Yapur	8083	pc_servicio	/api/v1/servicios
service-orden	Jael Yapur	8084	pc_orden	/api/v1/ordenes

Microservicio	Responsable	Puerto	Base de datos	Endpoint principal
service-voucher	Sarai Jara	8085	pc_voucher	/api/v1/vouchers
service-estadisticas	Felipe Moya	8086	pc_estadisticas	/api/v1/estadisticas
service-hardware	Felipe Moya	8087	pc_hardware	/api/v1/hardware
service-software	Jael Yapur	8088	pc_software	/api/v1/software
service-mantenimiento	Jael Yapur	8089	pc_mantenimiento	/api/v1/mantenimiento
service-descuento	Sarai Jara	8090	pc_descuento	/api/v1/descuento
service-auth	Sarai Jara	8091	pc_seguridad	/api/v1/auth

c. Servicio de autenticación

i. ¿En qué consiste la autenticación por token?

La autenticación por token es un mecanismo de seguridad que permite validar la identidad de un usuario antes de permitir el acceso a funcionalidades protegidas. El usuario inicia sesión con credenciales y, si estas son correctas, el sistema genera un token JWT. Ese token se utiliza posteriormente en cada petición protegida.

ii. ¿Cómo se implementa la autenticación por token?

En PCer2 se implementa mediante el microservicio service-auth y el filtro AuthenticationFilter en el API Gateway. service-auth registra usuarios, encripta contraseñas con BCrypt y genera un token JWT al iniciar sesión. El API Gateway valida el token antes de permitir el acceso a los demás microservicios.

```
POST http://localhost:8080/api/v1/auth/login
```

```
{
  "nombreUsuario": "admin",
  "password": "1234"
}
```

Respuesta esperada:
eyJhbGciOiJIUzI1NiJ9...

iii. Microservicio service-auth

El microservicio service-auth contiene las clases principales AutenticacionController, AuthService, JwtService, SecurityConfig, Usuario, Rol y UsuarioRepository. Su endpoint de login queda libre para permitir que el usuario obtenga un token sin necesitar un token previo.

iv. API Gateway y AuthenticationFilter

El API Gateway actúa como punto de entrada seguro. Las rutas de autenticación y documentación Swagger se dejan públicas, mientras que las rutas de los microservicios de negocio pasan por AuthenticationFilter. Si una petición no incluye token, el Gateway responde **"401 Unauthorized"**. Si el token es válido, la petición continúa hacia el microservicio correspondiente.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

v. Flujo de autenticación en PCer2

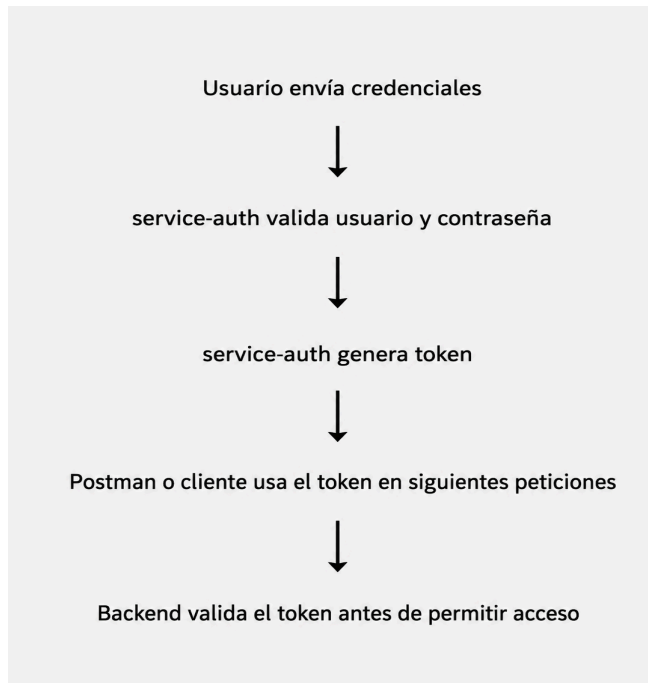
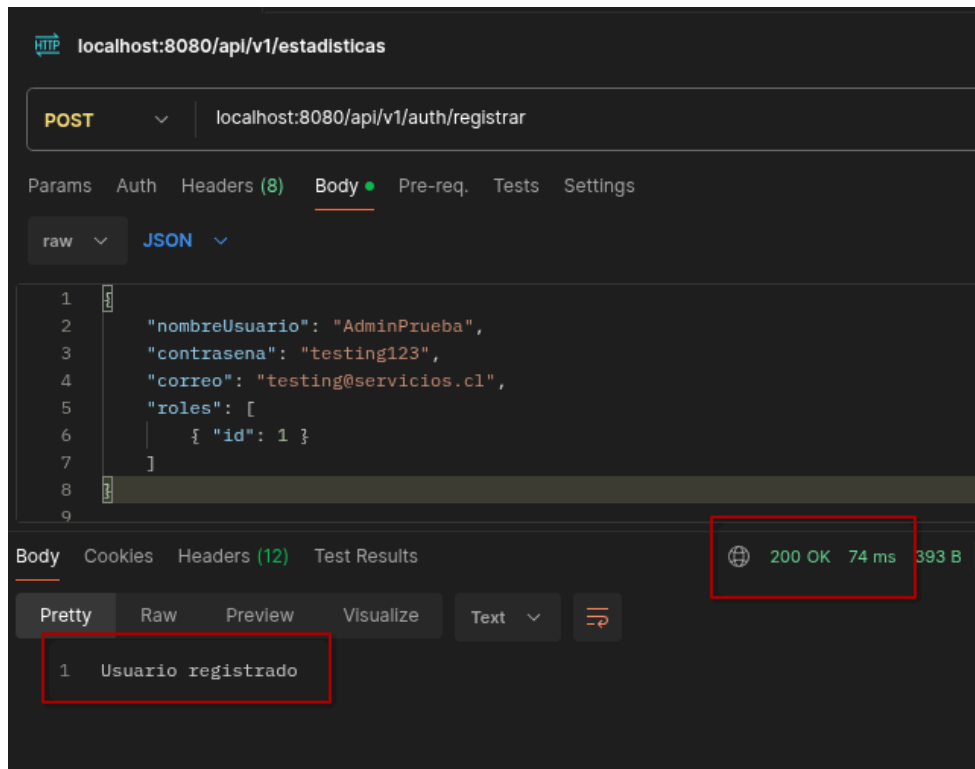


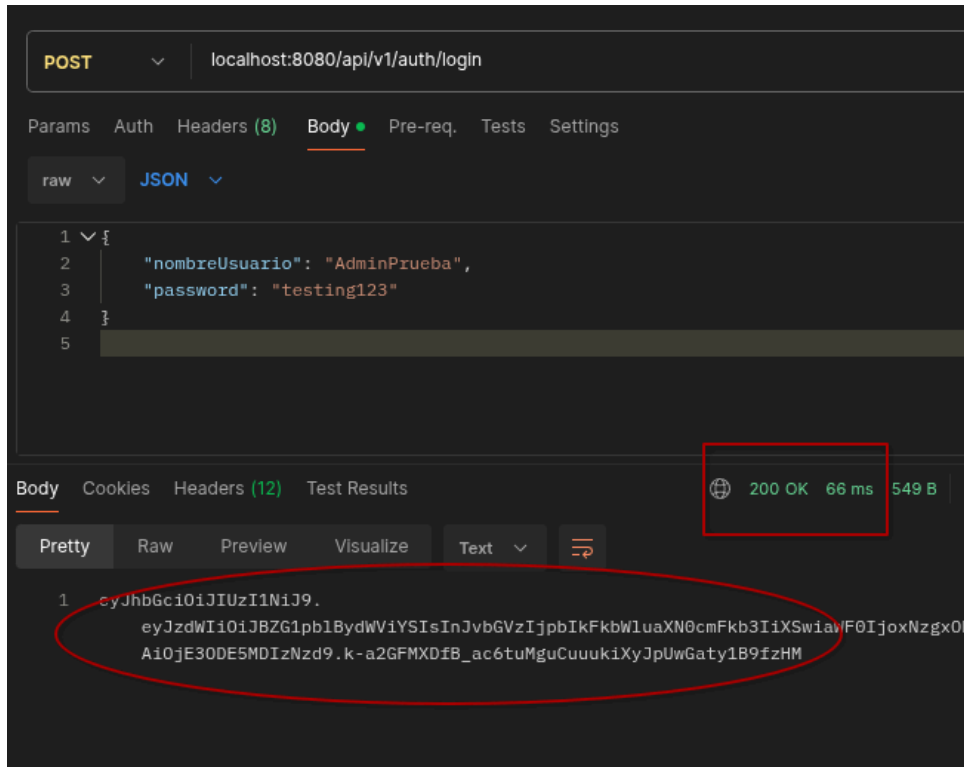
Figura 2. Flujo general de autenticación por token en PCer2.

- El usuario envía nombre de usuario y contraseña.
- service-auth valida las credenciales.
- Si los datos son correctos, service-auth genera un token JWT.
- Postman, Swagger o cliente web utiliza el token en las siguientes peticiones.
- El API Gateway valida el token antes de permitir el acceso.
- Si el token es inválido, faltante o expirado, responde 401 Unauthorized.

vi. Evidencias de autenticación



Registro de usuario en Postman.



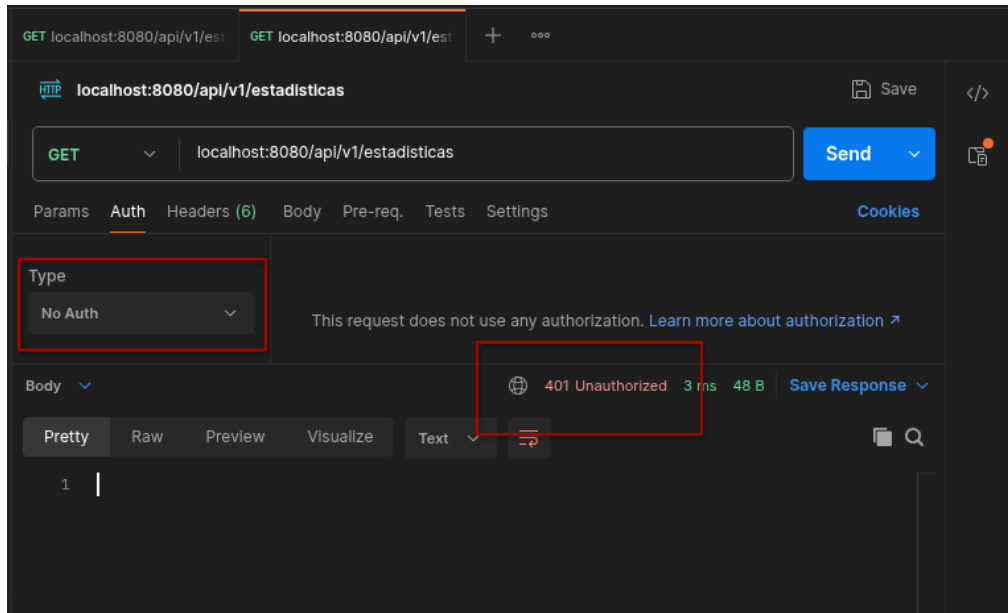
Login con generación de token JWT.

id	nombre_usuario	correo	contrasena
1	Pipe	asdfPipeee@gmail.com	\$2a\$10\$jkFyOc5.STkoTChH2DPRO2fZH1JaYvHqzNcyJI0yWY...
2	AdminPrueba	testing@servicios.cl	\$2a\$10\$qkYET2f/m2r4pb73Bzt6n.H/UXWknblinZbT.SaZzS2...

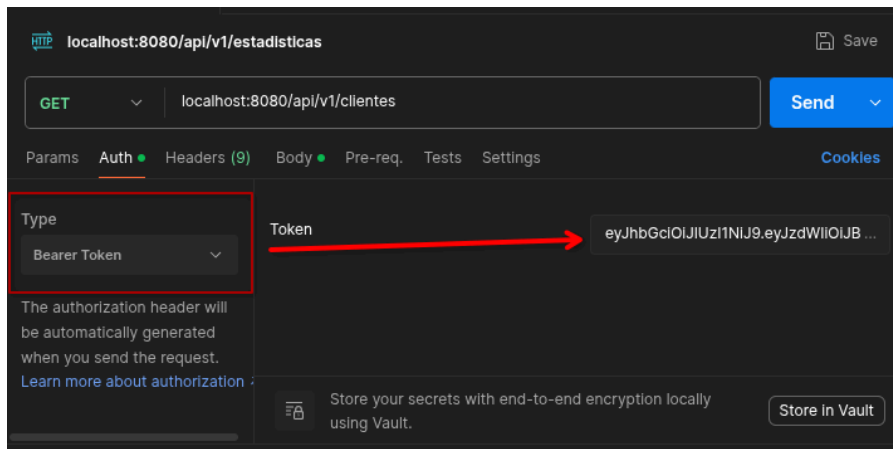
Para los elementos que están marcados: Editar Copiar Borrar Exportar

Orden de las columnas | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según

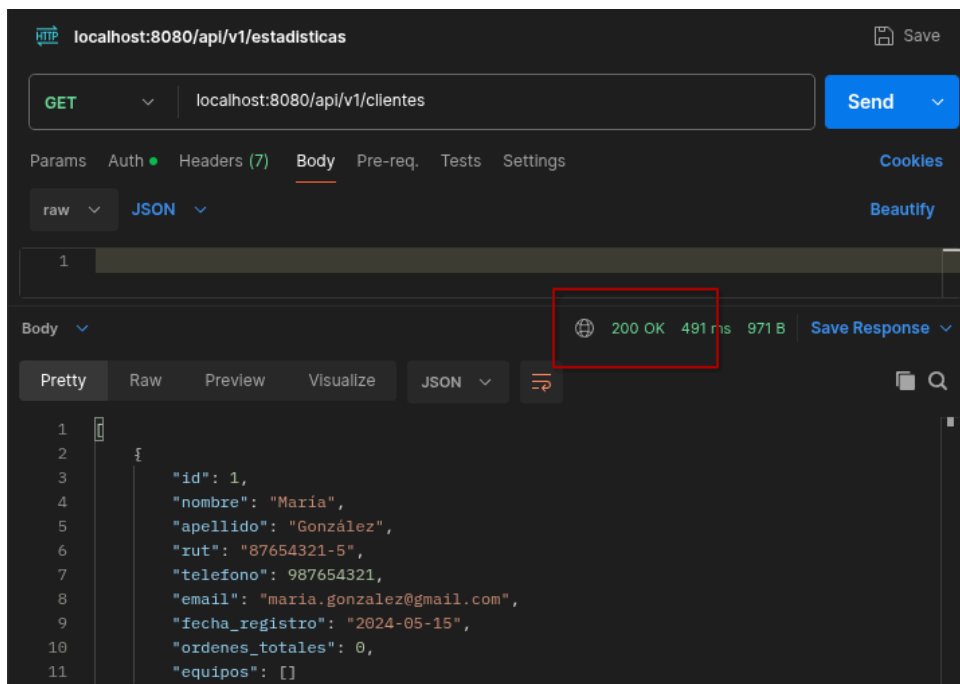
Tabla usuarios en MySQL con contraseña encriptada.



Endpoint protegido sin token respondiendo 401 Unauthorized.



Ingreso de bearer token previo al uso de un endpoint protegido



Endpoint protegido (service-clientes) con token respondiendo 200 OK.

```
J AutenticacionController.java 1, M X
service-auth > service-auth > src > main > java > com > pcer2 > service_auth > controller > J AutenticacionController.java > Language Support for Java(TM) by Red Hat

1 package com.pcer2.service_auth.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7 import com.pcer2.service_auth.dto.AuthRequest;
8 import com.pcer2.service_auth.model.Usuario;
9 import com.pcer2.service_auth.service.AuthService;
10 import io.swagger.v3.oas.annotations.Operation;
11 import io.swagger.v3.oas.annotations.tags.Tag;
12
13 @RestController
14 @RequestMapping("/api/v1/auth")
15 @CrossOrigin(origins = "**")
16 @Tag(name = "Autenticación", description = "Endpoints para registro y login de usuarios")
17 public class AutenticacionController {
18
19     @Autowired
20     private AuthService authService;
21
22     @PostMapping("/registrar")
23     @Operation(summary = "Registrar usuario", description = "Guarda un usuario con la contraseña encriptada")
24     public ResponseEntity<String> registrar(@RequestBody Usuario usuario) {
25         return ResponseEntity.ok(authService.registrar(usuario));
26     }
27
28     @PostMapping("/login")
29     @Operation(summary = "Iniciar sesión", description = "Retorna un token JWT si las credenciales son válidas")
30     public ResponseEntity<String> login(@RequestBody AuthRequest request) {
31         try {
32             String token = authService.login(request.getNombreUsuario(), request.getPassword());
33             return ResponseEntity.ok(token);
34         } catch (RuntimeException e) {
35             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(e.getMessage());
36         }
37     }
38 }
```

Código de AutenticacionController

```
J AuthService.java 3, M X
service-auth > service-auth > src > main > java > com > pcer2 > service_auth > service > J AuthService.java > Language Support for Java(TM) by Red Hat > AuthService

1 package com.pcer2.service_auth.service;
2
3 import java.util.List;
4 import java.util.stream.Collectors;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.security.crypto.password.PasswordEncoder;
7 import org.springframework.stereotype.Service;
8 import com.pcer2.service_auth.model.Rol;
9 import com.pcer2.service_auth.model.Usuario;
10 import com.pcer2.service_auth.repository.UsuarioRepository;
11
12 @Service
13 public class AuthService {
14
15     @Autowired
16     private UsuarioRepository usuarioRepo;
17
18     @Autowired
19     private JwtService jwtService;
20
21     @Autowired
22     private PasswordEncoder passwordEncoder;
23
24     public String registrar(Usuario usuario) {
25         usuario.setContraseña(passwordEncoder.encode(usuario.getContraseña()));
26         usuarioRepo.save(usuario);
27         return "Usuario registrado";
28     }
29
30     public String login(String username, String password) {
31         Usuario user = usuarioRepo.findByNombreUsuario(username)
32             .orElseThrow(() -> new RuntimeException(message: "Usuario no encontrado"));
33
34         if (passwordEncoder.matches(password, user.getContraseña())) {
35             List<String> roles = user.getRoles().stream()
36                 .map(Rol::getNombreRol)
37                 .collect(Collectors.toList());
38
39             return jwtService.generarToken(username, roles);
40         }
41
42         throw new RuntimeException(message: "Credenciales inválidas");
43     }
44 }
```

Código de AuthService

```
J JwtService.java X
service-auth > service-auth > src > main > java > com > pcer2 > service_auth > service > J JwtService.java > Language Support for Java(TM)

1 package com.pcer2.service_auth.service;
2
3 import java.util.Date;
4 import java.util.List;
5
6 import org.springframework.beans.factory.annotation.Value;
7 import org.springframework.stereotype.Service;
8
9 import io.jsonwebtoken.Jwts;
10 import io.jsonwebtoken.SignatureAlgorithm;
11 import io.jsonwebtoken.security.Keys;
12
13 @Service
14 public class JwtService {
15
16     @Value("${jwt.secret}")
17     private String secreto;
18
19     public String generarToken(String username, List<String> roles) {
20         long dosHorasEnMilisegundos = 1000 * 60 * 60 * 2;
21
22         return Jwts.builder()
23             .setSubject(username)
24             .claim(name: "roles", roles)
25             .setIssuedAt(new Date())
26             .setExpiration(new Date(System.currentTimeMillis() + dosHorasEnMilisegundos))
27             .signWith(Keys.hmacShaKeyFor(secreto.getBytes()), SignatureAlgorithm.HS256)
28             .compact();
29     }
30 }
```

Código de JwtService


```
J SecurityConfig.java X
service-auth > service-auth > src > main > java > com > pcer2 > service_auth > config > J SecurityConfig.java > Language Support for Java
1  package com.pcer2.service_auth.config;
2
3  import org.springframework.context.annotation.Bean;
4  import org.springframework.context.annotation.Configuration;
5  import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6  import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
7  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
8  import org.springframework.security.crypto.password.PasswordEncoder;
9  import org.springframework.security.web.SecurityFilterChain;
10
11 @Configuration
12 @EnableWebSecurity
13 public class SecurityConfig {
14
15     @Bean
16     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
17         return http.csrf(csrf -> csrf.disable())
18             .authorizeHttpRequests(auth -> auth
19                 .requestMatchers("/api/v1/auth/**").permitAll()
20                 .requestMatchers("/swagger-ui.html").permitAll()
21                 .requestMatchers("/swagger-ui/**").permitAll()
22                 .requestMatchers("/v3/api-docs/**").permitAll()
23                 .requestMatchers("/api/v1/auth/v3/api-docs/**").permitAll()
24                 .anyRequest().permitAll()
25             )
26             .build();
27     }
28
29     @Bean
30     public PasswordEncoder passwordEncoder() {
31         return new BCryptPasswordEncoder();
32     }
33 }
```

SecurityConfig

```
J AuthenticationFilter.java M X
api-gateway > api_gateway > src > main > java > com > pcer2 > api_gateway > config > J AuthenticationFilter.java > ...
1 package com.pcer2.api_gateway.config;
2
3 import java.nio.charset.StandardCharsets;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.cloud.gateway.filter.GatewayFilter;
6 import org.springframework.cloud.gateway.filter.factory.AbstractGatewayFilterFactory;
7 import org.springframework.http.HttpHeaders;
8 import org.springframework.http.HttpStatus;
9 import org.springframework.stereotype.Component;
10 import org.springframework.web.server.ServerWebExchange;
11 import io.jsonwebtoken.Jwts;
12 import io.jsonwebtoken.security.Keys;
13 import reactor.core.publisher.Mono;
14
15 @Component
16 public class AuthenticationFilter extends AbstractGatewayFilterFactory<AuthenticationFilter.Config> {
17
18     @Value("${jwt.secret:493358340235739058340598340598340598340598340598}")
19     private String secreto;
20
21     public AuthenticationFilter() {
22         super(configClass: Config.class);
23     }
24
25     public static class Config {
26         private String name;
27
28         public Config() {
29         }
30
31         public String getName() {
32             return name;
33         }
34
35         public void setName(String name) {
36             this.name = name;
37         }
38     }
39
40     @Override
41     public GatewayFilter apply(Config config) {
42         return (exchange, chain) -> {
43
44             String path = exchange.getRequest().getURI().getPath();
45
46             if (path.contains(s: "/v3/api-docs") ||
47                 path.contains(s: "/swagger-ui") ||
48                 path.contains(s: "/swagger-ui.html") ||
49                 path.startsWith(prefix: "/api/v1/auth")) {
50                 return chain.filter(exchange);
51             }
52
53             String authHeader = exchange.getRequest()
54                 .getHeaders()
55                 .getFirst(HttpHeaders.AUTHORIZATION);
56
57             if (authHeader == null || !authHeader.startsWith(prefix: "Bearer ")) {
58                 return onError(exchange, err: "Token faltante o formato inválido", HttpStatus.UNAUTHORIZED);
59             }
60
61             String token = authHeader.substring(beginIndex: 7);
62
63             try {
64                 Jwts.parserBuilder()
65                     .setSigningKey(Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8)))
66                     .build()
67                     .parseClaimsJws(token);
68             } catch (Exception e) {
69                 return onError(exchange, err: "Token inválido o expirado", HttpStatus.UNAUTHORIZED);
70             }
71
72             return chain.filter(exchange);
73         };
74     }
75
76     private Mono<Void> onError(ServerWebExchange exchange, String err, HttpStatus httpStatus) {
77         exchange.getResponse().setStatusCode(httpStatus);
78         return exchange.getResponse().setComplete();
79     }
80 }
81
```

Código de AuthenticationFilter.

d. Implementación por capas

i. Explicación general de la arquitectura por capas

Los microservicios del proyecto PCer2 están organizados por capas, lo que permite separar responsabilidades, mantener el código ordenado y facilitar el testing unitario.

ii. Capa Controller

Recibe las solicitudes HTTP desde Postman, Swagger, frontend o API Gateway. Por ejemplo, `LicenciaSoftwareController` expone endpoints para crear, listar, buscar, actualizar y eliminar licencias de software.

iii. Capa DTO

Representa los datos que llegan desde una petición. Permite separar los datos de entrada de la entidad real que se guarda en la base de datos.

iv. Capa Service

Contiene la lógica de negocio. Convierte DTO a Entity, coordina validaciones y llama al Repository. En `service-orden` también coordina la comunicación con otros microservicios mediante `WebClient`.

v. Capa Repository

Se comunica con la base de datos mediante JPA. Permite utilizar métodos como `save()`, `findAll()`, `findById()` y `deleteById()`.

vi. Capa Model / Entity

Representa las entidades persistidas en base de datos. Cada Entity normalmente corresponde a una tabla.

vii. Ejemplo de implementación por capas usando service-software

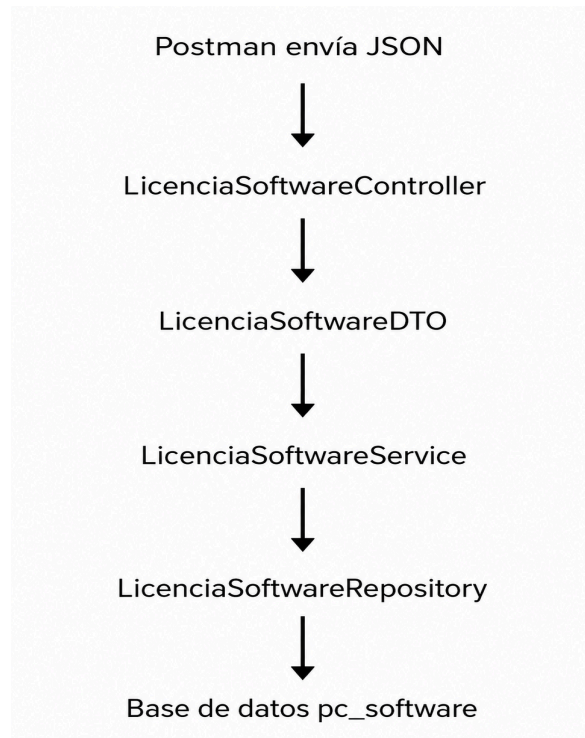


Figura 3. Flujo por capas en service-software.

```
POST /api/v1/software
```

```
{
  "nombre": "Windows 11 Pro",
  "marca": "Microsoft",
  "version": "11 Pro",
  "serial": "WIN11-PRO-ABC123"
}
```

viii. Evidencias de código

```
J LicenciaSoftwareController.java 1, M X
service-software > service_software > src > main > java > com > pcer2 > service_software > controller > J LicenciaSoftwareController.java > Language Support for Java(TM) by Red Hat > LicenciaSoftware
12
13 @RestController
14 @RequestMapping("/api/v1/software")
15 @CrossOrigin(origins = "")
16 //permitir que Swagger pueda llamar al Controller desde otro puerto
17 @Tag(name = "Gestión de Software", description = "Endpoints para administrar licencias de software")
18 public class LicenciaSoftwareController {
19
20     @Autowired
21     private LicenciaSoftwareService licenciaSoftwareService;
22
23     @GetMapping
24     @Operation(summary = "Listar licencias de software", description = "Obtiene todas las licencias de software registradas en el sistema")
25     public List<LicenciaSoftware> listar() {
26         return licenciaSoftwareService.listarTodos();
27     }
28
29     @PostMapping
30     @Operation(summary = "Crear licencia de software", description = "Registra una nueva licencia de software en el sistema")
31     public ResponseEntity<LicenciaSoftware> crear(@RequestBody LicenciaSoftwareDTO licenciaSoftwareDTO) {
32         return ResponseEntity.ok(licenciaSoftwareService.guardar(licenciaSoftwareDTO));
33     }
34
35     @GetMapping("/{id}")
36     @Operation(summary = "Buscar licencia por ID", description = "Obtiene una licencia de software específica según su ID")
37     public ResponseEntity<LicenciaSoftware> obtenerPorId(@PathVariable Long id) {
38         return licenciaSoftwareService.buscarPorId(id)
39             .map(ResponseEntity::ok)
40             .orElse(ResponseEntity.notFound().build());
41     }
42
43     @PutMapping("/{id}")
44     @Operation(summary = "Actualizar licencia de software", description = "Actualiza los datos de una licencia de software existente")
45     public ResponseEntity<LicenciaSoftware> actualizar(@PathVariable Long id, @RequestBody LicenciaSoftwareDTO licenciaSoftwareDTO) {
46         LicenciaSoftware licenciaActualizada = licenciaSoftwareService.actualizarLicencia(id, licenciaSoftwareDTO);
47
48         if (licenciaActualizada != null) {
49             return ResponseEntity.ok(licenciaActualizada);
50         }
51
52         return ResponseEntity.notFound().build();
53     }
54
55     @DeleteMapping("/{id}")
56     @Operation(summary = "Eliminar licencia de software", description = "Elimina una licencia de software según su ID")
57     public ResponseEntity<Void> eliminar(@PathVariable Long id) {
58         licenciaSoftwareService.eliminarLicencia(id);
59         return ResponseEntity.noContent().build();
60     }
61 }
```

LicenciaSoftwareController: recibe la petición y responde HTTP.

```
J LicenciaSoftwareDTO.java X
service-software > service_software > src > main > java > com > pcer2 > service_software > dto > J LicenciaSoftwareDTO.java > {}
1 package com.pcer2.service_software.dto;
2
3 import io.swagger.v3.oas.annotations.media.Schema;
4 import lombok.AllArgsConstructor;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7
8 @Data
9 @AllArgsConstructor
10 @NoArgsConstructor
11 @Schema(description = "DTO utilizado para crear o actualizar una licencia de software")
12 public class LicenciaSoftwareDTO {
13
14     @Schema(description = "Nombre del software", example = "Windows 11 Pro")
15     private String nombre;
16
17     @Schema(description = "Marca o proveedor del software", example = "Microsoft")
18     private String marca;
19
20     @Schema(description = "Versión del software", example = "11 Pro")
21     private String version;
22
23     @Schema(description = "Serial único de la licencia", example = "WIN11-PRO-ABC123")
24     private String serial;
25 }
```

LicenciaSoftwareDTO: transporta los datos enviados desde Postman.

```
J LicenciaSoftwareService.java 1, M X
service-software > service_software > src > main > java > com > pcer2 > service_software > service > J LicenciaSoftwareService.java > ...
1 package com.pcer2.service_software.service;
2
3 import java.util.List;
4 import java.util.Optional;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7 import com.pcer2.service_software.dto.LicenciaSoftwareDTO;
8 import com.pcer2.service_software.model.LicenciaSoftware;
9 import com.pcer2.service_software.repository.LicenciaSoftwareRepository;
10
11 @Service
12 public class LicenciaSoftwareService {
13
14     @Autowired
15     private LicenciaSoftwareRepository licenciaSoftwareRepository;
16
17     public LicenciaSoftware guardar(LicenciaSoftwareDTO licenciaSoftwareDTO) {
18
19         LicenciaSoftware licenciaSoftware = new LicenciaSoftware();
20
21         licenciaSoftware.setNombre(licenciaSoftwareDTO.getNombre());
22         licenciaSoftware.setMarca(licenciaSoftwareDTO.getMarca());
23         licenciaSoftware.setVersion(licenciaSoftwareDTO.getVersion());
24         licenciaSoftware.setSerial(licenciaSoftwareDTO.getSerial());
25
26         return licenciaSoftwareRepository.save(licenciaSoftware);
27     }
28
29     public List<LicenciaSoftware> listarTodos() {
30         return licenciaSoftwareRepository.findAll();
31     }
32
33     public Optional<LicenciaSoftware> buscarPorId(Long id) {
34         return licenciaSoftwareRepository.findById(id);
35     }
36
37     public LicenciaSoftware actualizarLicencia(Long id, LicenciaSoftwareDTO licenciaSoftwareDTO) {
38
39         Optional<LicenciaSoftware> licenciaExistente = licenciaSoftwareRepository.findById(id);
40
41         if (licenciaExistente.isPresent()) {
42
43             LicenciaSoftware licenciaSoftware = licenciaExistente.get();
44
45             licenciaSoftware.setNombre(licenciaSoftwareDTO.getNombre());
46             licenciaSoftware.setMarca(licenciaSoftwareDTO.getMarca());
47             licenciaSoftware.setVersion(licenciaSoftwareDTO.getVersion());
48             licenciaSoftware.setSerial(licenciaSoftwareDTO.getSerial());
49
50             return licenciaSoftwareRepository.save(licenciaSoftware);
51         }
52
53         return null;
54     }
55
56     public void eliminarLicencia(Long id) {
57         licenciaSoftwareRepository.deleteById(id);
58     }
59 }
```

LicenciaSoftwareService: contiene la lógica de guardado, listado, búsqueda, actualización y eliminación.

```
J LicenciaSoftwareRepository.java 1 X
service-software > service_software > src > main > java > com > pcer2 > service_software > repository > J LicenciaSoftwareRepository.java
1 package com.pcer2.service_software.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.stereotype.Repository;
5
6 import com.pcer2.service_software.model.LicenciaSoftware;
7
8 @Repository
9 public interface LicenciaSoftwareRepository extends JpaRepository<LicenciaSoftware, Long> {
10
11 }
```

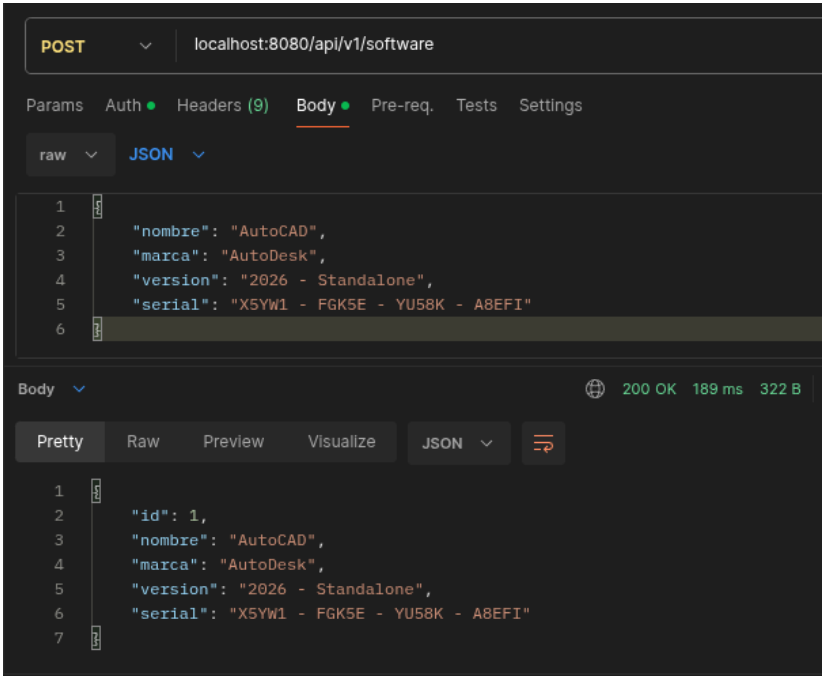
LicenciaSoftwareRepository: hereda de JpaRepository.

```
J LicenciaSoftware.java M X
service-software > service_software > src > main > java > com > pcer2 > service_software > model > J LicenciaSoftware.java > LicenciaSoftware
1 package com.pcer2.service_software.model;
2
3 import io.swagger.v3.oas.annotations.media.Schema;
4 import jakarta.persistence.*;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 @Entity
10 @Data
11 @Table(name = "licencias_software")
12 @AllArgsConstructor
13 @NoArgsConstructor
14 @Schema(description = "Entidad que representa una licencia de software registrada en el sistema")
15 public class LicenciaSoftware {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     @Schema(description = "ID único de la licencia de software", example = "1")
20     private Long id;
21
22     @Schema(description = "Nombre del software", example = "Windows 11 Pro")
23     private String nombre;
24
25     @Schema(description = "Marca o proveedor del software", example = "Microsoft")
26     private String marca;
27
28     @Schema(description = "Versión del software", example = "11 Pro")
29     private String version;
30
31     @Column(unique = true)
32     @Schema(description = "Serial único de la licencia", example = "WIN11-PRO-ABC123")
33     private String serial;
34 }
```

LicenciaSoftware: entidad que representa la tabla `licencias_software`.

ix. Evidencias en Postman: POST, GET, GET(id), PUT(id), DELETE(id)

Método	Ruta	Objetivo
POST	http://localhost:8080/api/v1/software	Crear licencia de software usando API Gateway y token.
GET	http://localhost:8080/api/v1/software	Listar licencias registradas.
GET(id)	http://localhost:8080/api/v1/software/1	Buscar licencia por ID.
PUT(id)	http://localhost:8080/api/v1/software/1	Actualizar licencia existente.
DELETE(id)	http://localhost:8080/api/v1/software/1	Eliminar licencia por ID.



[POST] Creación de licencia de software


```
GET localhost:8080/api/v1/software

Params Auth Headers (9) Body Pre-req. Tests Settings
raw JSON
1
2 "nombre": "Office",
3 "marca": "Microsoft",
4
5 {
6   "id": 1,
7   "nombre": "AutoCAD",
8   "marca": "AutoDesk",
9   "version": "2026 - Standalone",
10  "serial": "X5YW1 - FGK5E - YU58K - A8EFI"
11 },
12 {
13   "id": 2,
14   "nombre": "Office",
15   "marca": "Microsoft",
16   "version": "2024 Pro Plus",
17   "serial": "57892 - 27864 - 72484 - 33657"
18 }
19
20 Body 200 OK 70 ms 438 B
```

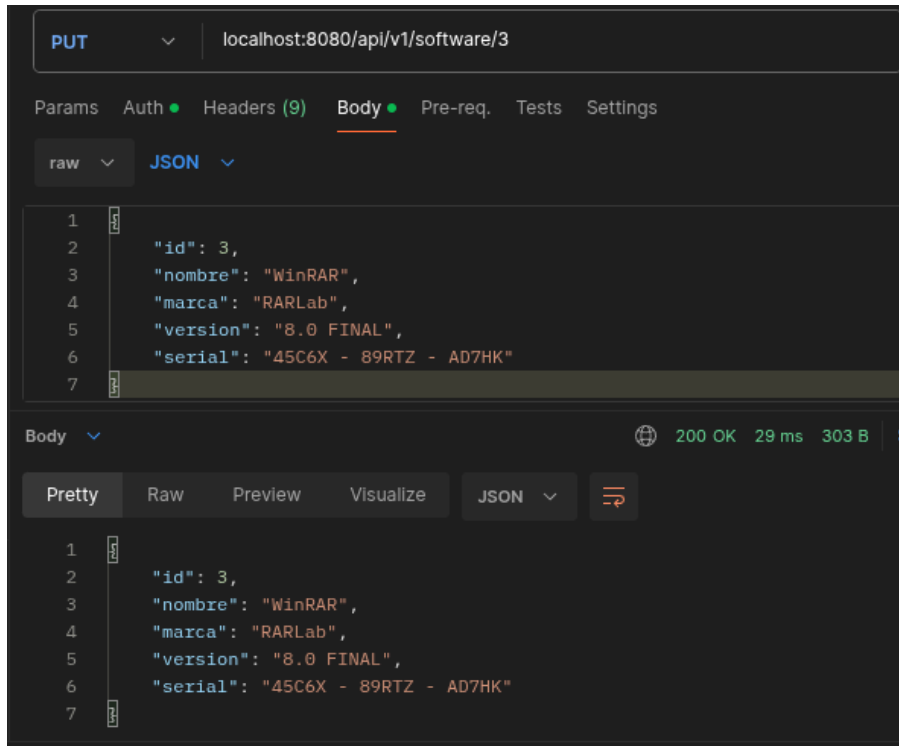
[GET] Se listan 2 licencias previamente creadas

```
GET localhost:8080/api/v1/software/3

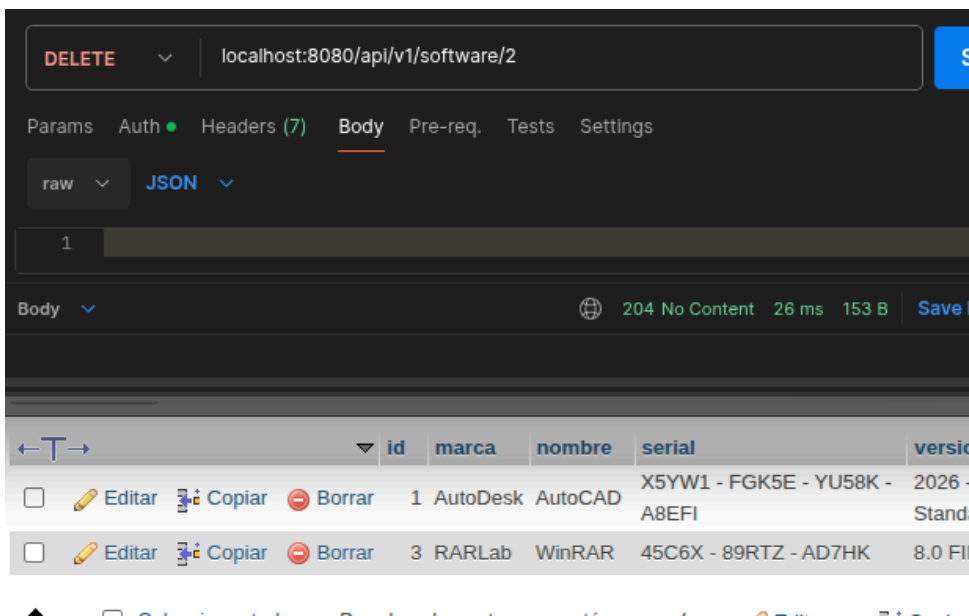
Params Auth Headers (7) Body Pre-req. Tests Settings
raw JSON
1

2 {
3   "id": 3,
4   "nombre": "WinRAR",
5   "marca": "RARLab",
6   "version": "7.22",
7   "serial": "A485W - 33X5N - E845D"
8 }
9
10 Body 200 OK 17 ms 298 B
```

[GET (id)] Búsqueda del software ID = 3



[PUT (id)] Modificación del software ID = 3



[DELETE (id)] Eliminación de software ID = 2.

Se observa en el gestor de BD que ya no existe dicho ID.

x. Comunicación entre microservicios

La comunicación entre microservicios se implementa mediante peticiones HTTP. El ejemplo principal es service-orden, que guarda identificadores de otros microservicios y luego consulta datos externos mediante WebClient para mostrar una respuesta más completa.

xi. Ejemplo de comunicación mediante WebClient en service-orden

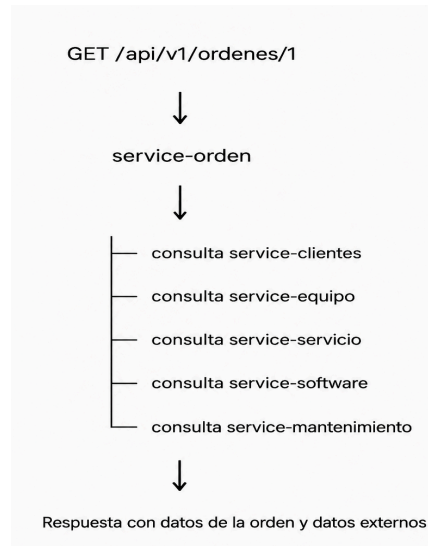


Figura 4. Comunicación de service-orden con otros microservicios.

```

webClientBuilder.build()
    .get()
    .uri("http://localhost:8088/api/v1/software/" + ordenTrabajo.getSoftwareId())
    .retrieve()
    .bodyToMono(Object.class)
    .block();
  
```

Los datos externos se manejan con campos @Transient, por lo que se muestran en la respuesta pero no se guardan en la tabla de órdenes.

```

1  GET localhost:8080/api/v1/ordenes/1
2
3  Params Authorization Headers (7) Body Pre-request Script Tests Settings
4
5  Body Cookies Headers (6) Test Results
6
7  Pretty Raw Preview Visualize JSON
8
9  {
10   "id": 1,
11   "clienteId": 1,
12   "equipoId": 1,
13   "servicioId": 3,
14   "softwareId": null,
15   "productoMantenimientoId": 1,
16   "descripcionProblema": "Equipo muy sucio. Presenta estrangulamiento térmico en GPU y CPU.",
17   "fechaIngreso": "2026-05-15",
18   "fechaEntregaEstimada": "2026-05-18",
19   "estado": "Estética 8/10 - Funcionalidad 4/10",
20   "precioTotal": 35000.0,
21   "datosCliente": {
22     "rut": "87654321-5",
23     "nombre": "Maria",
24     "apellido": "González"
25   },
26   "datosEquipo": {
27     "id": 1,
28     "tipoEquipo": "Notebook",
29     "marca": "Lenovo",
30     "modelo_cpu": "Ryzen 5 8540U",
31     "placa_madre": "MS7989",
32     "ram_cantidad": 32,
33     "ram_frecuencia": 5800,
34     "almacen_cantidad": 2048,
35     "tipoAlmacen": "SSD",
36     "numeroserie": "21CR0006US",
37     "veces_reparado": 0,
38     "clienteId": 3
39   },
40   "datosServicio": {
41     "nombre": "Mantención completa de equipo"
42   },
43   "datosSoftware": null,
44   "datosProductoMantenimiento": {
45     "id": 1,
46     "nombre": "Pasta Termica Arctic Silver 5",
47     "categoria": "Solución Térmica",
48     "descripcion": "Alta Densidad 2,7gr. Pasta Disipadora.",
49     "stockActual": 3,
50     "precioUnitario": 5990.0
51   }
52 }

```

Ejemplo de orden de trabajo que muestra conexión con diferentes microservicios

e. Swagger / OpenAPI

i. ¿Qué es Swagger?

Swagger es un conjunto de herramientas que permite documentar, visualizar y probar APIs REST de forma interactiva. En Spring Boot se puede implementar mediante SpringDoc OpenAPI, que genera documentación automática a partir de los Controllers, DTOs y Models del proyecto.

ii. ¿En qué consiste Swagger?

Swagger permite visualizar la documentación de una API REST de forma clara e interactiva. Muestra los endpoints disponibles, los métodos HTTP como **GET**, **POST**, **PUT** y **DELETE**, los parámetros, el **request body**, las respuestas esperadas y los modelos o esquemas de datos. Además, permite probar los endpoints directamente desde el navegador mediante Swagger UI.

iii. ¿Cómo se implementa Swagger en microservicios Spring Boot?

En cada microservicio MVC se agrega la dependencia springdoc-openapi-starter-webmvc-ui. En API Gateway, al trabajar con WebFlux, se utiliza springdoc-openapi-starter-webflux-ui.

iv. Dependencias utilizadas

```
<!-- Dependencia para microservicios Spring MVC -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.16</version>
</dependency>

<!-- Dependencia para API Gateway con WebFlux -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webflux-ui</artifactId>
  <version>2.8.16</version>
</dependency>
```

v. Anotaciones utilizadas: @Tag, @Operation y @Schema

Anotación	Uso
@Tag	Agrupar endpoints dentro de Swagger.
@Operation	Describe qué hace cada endpoint.
@Schema	Describe entidades, DTOs y atributos mostrados en la documentación.

vi. OpenApiConfig y token Bearer en Swagger

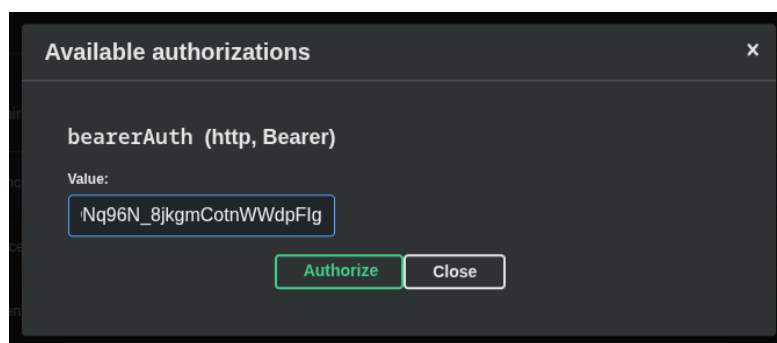
OpenApiConfig permite configurar Swagger/OpenAPI para documentar el uso de autenticación mediante **Bearer Token con JWT**. Gracias a esta configuración, Swagger muestra el botón **Authorize**, donde se puede ingresar el token y probar endpoints protegidos desde el navegador.

Este archivo **no valida el token por sí solo**. La validación real del JWT la realiza el **AuthenticationFilter** en el API Gateway. Su objetivo principal es facilitar la documentación y prueba de endpoints protegidos.

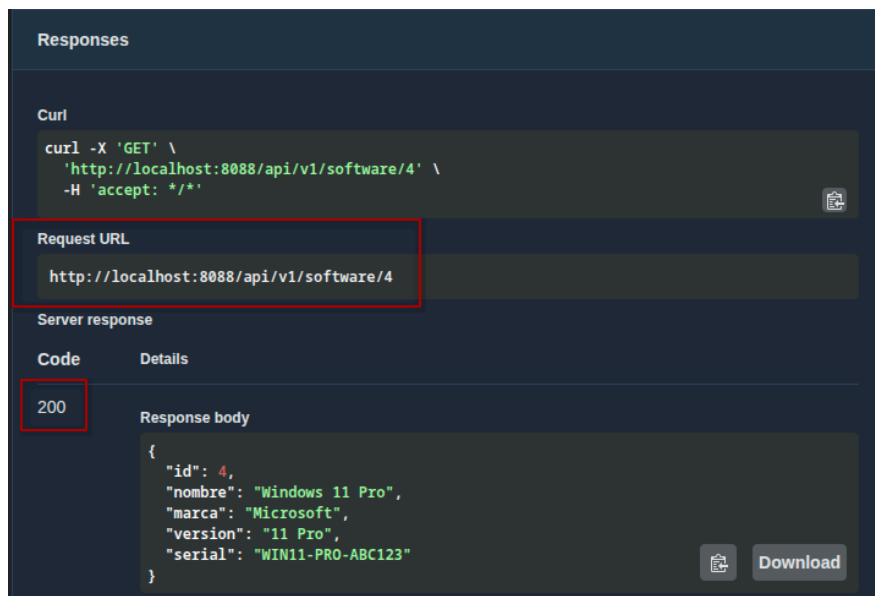
Está súper bien, solo cuida escribirlo como **OpenAPIConfig** o **OpenApiConfig** según el nombre exacto de tu clase en el proyecto.

vii. Ejemplo con evidencias en 3 microservicios

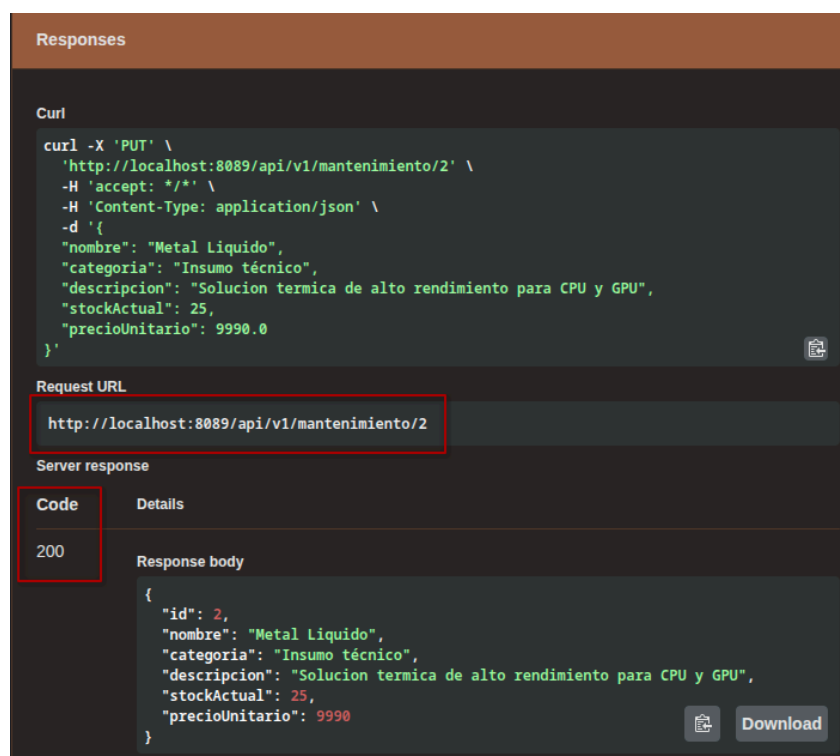
Microservicio	Swagger individual
service-software	http://localhost:8088/swagger-ui.html
service-mantenimiento	http://localhost:8089/swagger-ui.html
service-orden	http://localhost:8084/swagger-ui.html



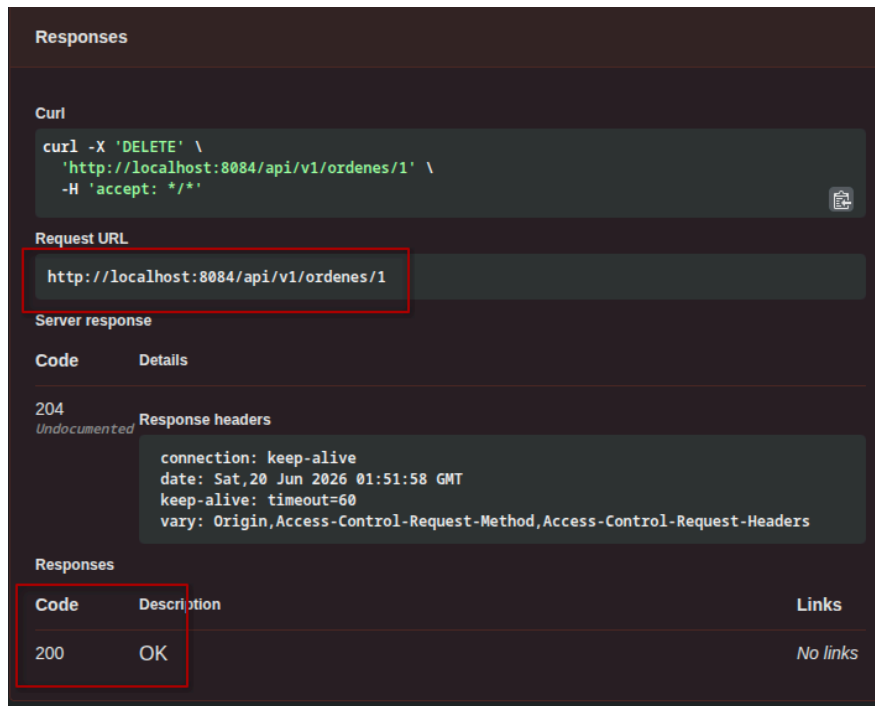
Ingreso de BEARER en Swagger



Prueba de [GET] ID = 4 en SwaggerUI directamente en endpoint de Software y usando autenticación



Prueba de [PUT] ID = 2 en SwaggerUI directamente en endpoint de Mantenimiento y usando autenticación



Prueba de [DELETE] ID = 1 en SwaggerUI directamente en endpoint de Ordenes y usando autenticación

viii. Swagger centralizado en API Gateway

Además de Swagger individual, se configuró Swagger centralizado desde el API Gateway. Desde <http://localhost:8080/swagger-ui.html> se puede seleccionar la documentación de los microservicios configurados en el gateway.

A continuación, se muestra un fragmento de la configuración del Swagger centralizado, donde se observan algunos microservicios registrados como ejemplo:

```
springdoc:
  swagger-ui:
    urls:
      - name: Software
        url: /api/v1/software/v3/api-docs
      - name: Mantenimiento
        url: /api/v1/mantenimiento/v3/api-docs
      - name: Auth
        url: /api/v1/auth/v3/api-docs
```


f. Testing

i. ¿En qué consiste el testing?

El testing consiste en ejecutar pruebas sobre el código para verificar que los métodos y funcionalidades implementadas se comporten de forma correcta.

ii. ¿Por qué es importante testear?

- Detecta errores antes de la entrega.
- Valida la lógica del sistema.
- Evita depender solamente de pruebas manuales.
- Facilita futuras modificaciones.
- Aumenta la calidad y confiabilidad del proyecto.

iii. Pruebas unitarias

Las pruebas unitarias validan unidades pequeñas de código, como métodos de una clase Service. En PCer2 se probaron métodos de guardar, listar, buscar por ID, actualizar, eliminar y, en service-orden, comunicación externa simulada mediante WebClient.

iv. Patrón AAA: Arrange, Act, Assert

Etapas	Significado	Explicación
Arrange	Preparar	Se crean datos de prueba y se configuran mocks.
Act	Actuar	Se ejecuta el método que se quiere probar.
Assert	Verificar	Se comprueba que el resultado sea el esperado.

v. Testing con JUnit 5 y Mockito

JUnit 5 se utilizó para crear y ejecutar pruebas unitarias. Mockito se utilizó para simular dependencias como Repository, JwtService, PasswordEncoder y WebClient. Así se prueba la lógica del Service sin conectarse a una base de datos real ni levantar otros microservicios.

vi. Testing implementado por microservicio

Microservicio	Responsable	Testing documentado
service-clientes	Felipe Moya	JUnit 5 + Mockito
service-equipo	Felipe Moya	JUnit 5 + Mockito
service-estadísticas	Felipe Moya	JUnit 5 + Mockito
service-hardware	Felipe Moya	JUnit 5 + Mockito

Microservicio	Responsable	Testing documentado
service-servicio	Jael Yapur	JUnit 5 + Mockito
service-orden	Jael Yapur	JUnit 5 + Mockito + WebClient simulado
service-software	Jael Yapur	JUnit 5 + Mockito
service-mantenimiento	Jael Yapur	JUnit 5 + Mockito
service-auth	Jael Yapur	JUnit 5 + Mockito
service-voucher	Sarai Jara	JUnit 5 + Mockito
service-descuento	Sarai Jara	JUnit 5 + Mockito

vii. Documentación del 60% del testing implementado

Para cumplir con el 60% solicitado, el proyecto incorpora pruebas unitarias en los microservicios principales utilizando JUnit 5 y Mockito. En el informe se documentan evidencias de 9 microservicios sobre un total de 11: service-clientes, service-equipo, service-estadisticas, service-hardware, service-servicio, service-orden, service-software, service-mantenimiento y service-auth.

Esto representa más del 60% del proyecto y permite evidenciar que se probaron métodos relevantes como guardar, listar, buscar por ID, actualizar, eliminar y validar casos específicos de negocio. Los microservicios service-voucher y service-descuento también se consideran dentro del proyecto, pero no se documentan en detalle en esta sección para mantener el informe enfocado en el porcentaje solicitado.

viii. Evidencias de métodos testeados

- **service-software:** Guardar, listar, buscar por ID, actualizar y eliminar.
- **service-mantenimiento:** Guardar, listar, buscar por ID, actualizar y eliminar.
- **service-servicio:** Guardar, listar, buscar por ID, actualizar y eliminar.
- **service-orden:** Guardar, listar, buscar por ID, actualizar, eliminar y simular WebClient.
- **service-auth:** Registrar usuario, login correcto, usuario no encontrado y credenciales inválidas.
- **service-voucher:** Guardar, listar, buscar por ID, actualizar y eliminar.
- **service-descuento:** Guardar, listar, buscar por ID, buscar por código y eliminar.

Servicios de Felipe: Tests unitarios de clientes, equipos, estadísticas y hardware.

❖ Test unitario de clientes

```

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  RESULTADOS DE LA PRUEBA  PUERTOS

%TESTS 8, buscarPorRutTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTE 8, buscarPorRutTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTS 9, buscarPorIdExistenteTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTE 9, buscarPorIdExistenteTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTS 10, eliminarClienteTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTE 10, eliminarClienteTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTS 11, guardarTest(com.pcer2.clientes.service.ClienteServiceTest)
%TESTE 11, guardarTest(com.pcer2.clientes.service.ClienteServiceTest)

%RUNTIME3262
  
```

- **contextLoads:** Valida que la aplicación de Spring Boot sea capaz de arrancar y cargar su configuración básica sin lanzar errores.
- **guardar y save:** Valida el registro o persistencia de un cliente.
- **listarTodos:** Valida el flujo de lectura masiva. Asegura que el método devuelva la lista completa de clientes registrados cuando se solicita.
- **buscarPorIdExistente:** Verifica que si el ID existe en la base de datos, el servicio devuelve correctamente el objeto del cliente correspondiente.
- **buscarPorIdExistente:** Verifica el comportamiento del sistema cuando alguien busca un ID que no está registrado.
- **buscarPorRut:** Verifica que la lógica filtre e identifique de manera exacta a un cliente a partir de su documento único.
- **eliminarCliente:** Comprueba que al pasarle el ID de un cliente, el servicio interactúe correctamente con el repositorio para eliminarlo del sistema.

❖ Test unitario de equipos

```

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  RESULTADOS DE LA PRUEBA  PUERTOS

%TESTS 12, buscarPorMarcaTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTE 12, buscarPorMarcaTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTS 13, guardarTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTE 13, guardarTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTS 14, eliminarNoExistenteTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTE 14, eliminarNoExistenteTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTS 15, findByIdNoExistenteTest(com.pcer2.service_equipo.service.EquipoServiceTest)
%TESTE 15, findByIdNoExistenteTest(com.pcer2.service_equipo.service.EquipoServiceTest)

%RUNTIME3024
  
```

- **listarTodos:** Verifica que el método para obtener todos los equipos retorne la lista correctamente.
- **guardar:** Comprueba que un equipo se guarde correctamente en la base de datos.
- **findByIdExistente:** Verifica que al buscar un ID existente se retorne el equipo correspondiente.
- **findByIdNoExistente:** Verifica el comportamiento cuando el ID no existe.
- **buscarPorNumeroSerie:** Comprueba que la búsqueda por número de serie funcione correctamente.
- **buscarPorMarca:** Verifica que se obtengan los equipos de una marca determinada.
- **findByClienteldSinEquipos:** Comprueba el comportamiento cuando un cliente no tiene equipos asociados.
- **findByClienteld:** Verifica que se obtengan correctamente los equipos de un cliente específico.
- **existePorNumeroSerie:** Comprueba que el método que valida la existencia de un número de serie retorne true o false según corresponda.
- **eliminarExistente:** Verifica que se pueda eliminar un equipo existente.
- **eliminarNoExistente:** Comprueba el comportamiento al intentar eliminar un equipo que no existe.

❖ Test unitario de estadísticas

```

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  RESULTADOS DE LA PRUEBA  PUERTOS
%TESTS  7, listarTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTE  7, listarTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTS  8, eliminarTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTE  8, eliminarTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTS  9, buscarNoExisteTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTE  9, buscarNoExisteTest(com.pcer2.service_estadisticas.service.EstadisticasServiceTest)
%TESTS  11, contextLoads(com.pcer2.service_estadisticas.ServiceEstadisticasApplicationTests)
%TESTE  11, contextLoads(com.pcer2.service_estadisticas.ServiceEstadisticasApplicationTests)
%RUNTIME3323
  
```

Test Runner for Java

- ✓ EstadisticasServiceTest \$(symbol-namespace) com.pcer2.service_esta...
- ✓ ServiceEstadisticasApplicationTests \$(symbol-namespace) com.pcer2...
- ✓ buscarNoExisteTest() \$(symbol-class) EstadisticasServiceTest + \$(symb...
- ✓ buscarTest() \$(symbol-class) EstadisticasServiceTest + \$(symbol-namesp...
- ✓ eliminarTest() \$(symbol-class) EstadisticasServiceTest + \$(symbol-names...
- ✓ generarReporteClientesTest() \$(symbol-class) EstadisticasServiceTest ...
- ✓ generarReporteEquiposTest() \$(symbol-class) EstadisticasServiceTest ...
- ✓ generarReporteVoucherTest() \$(symbol-class) EstadisticasServiceTest ...
- ✓ listarTest() \$(symbol-class) EstadisticasServiceTest + \$(symbol-namespac...
- ✓ contextLoads() \$(symbol-class) ServiceEstadisticasApplicationTests + \$(s...

> 6 resultados anteriores

- **buscar:** Verifica que se pueda buscar una estadística existente por su ID y que retorne los datos correctos.
- **buscarNoExiste:** Comprueba que al buscar una estadística que no existe se maneje correctamente el caso.
- **listar:** Verifica que el método que lista todas las estadísticas retorne la colección esperada.
- **eliminar:** Comprueba que una estadística existente pueda eliminarse correctamente.

- **generarReporteClientes:** Verifica que el reporte de clientes se genere correctamente y contenga la información esperada.
- **generarReporteEquipos:** Comprueba la generación del reporte de equipos.
- **generarReporteVoucher:** Verifica la generación del reporte de vouchers.
- **contextLoads:** Comprueba que Spring Boot pueda iniciar el contexto de la aplicación sin errores de configuración.

❖ Test unitario de hardware

```

SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  RESULTADOS DE LA PRUEBA  PUERTOS
%TESTS  10.guardarTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTE  10.guardarTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTS  11.eliminarReduciendoCantidadTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTE  11.eliminarReduciendoCantidadTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTS  12.buscarPorIdTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTE  12.buscarPorIdTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTS  13.eliminarHardwareTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%TESTE  13.eliminarHardwareTest(com.pcer2.serviceHardware.service.HardwareServiceTest)
%RUNTIME3027
pi_gateway> (pcer2-yapur-jara-moya-010D)  Java: Ready
  
```

- **listarTodos:** Verifica que se obtenga correctamente la lista de todos los componentes de hardware almacenados.
- **guardar:** Comprueba que un hardware nuevo pueda guardarse correctamente en la base de datos.
- **buscarPorId:** Verifica que al buscar un ID existente se retorne el hardware correcto.
- **buscarPorIdNoExiste:** Comprueba el comportamiento cuando se busca un hardware que no existe.
- **actualizar:** Verifica que los datos de un hardware existente puedan modificarse correctamente.
- **actualizarNoExiste:** Comprueba que no se pueda actualizar un hardware inexistente y que se maneje el error adecuadamente.
- **eliminarHardware:** Verifica que un hardware existente pueda eliminarse correctamente.
- **eliminarHardwareNoExiste:** Comprueba el comportamiento al intentar eliminar un hardware que no existe.
- **eliminarReduciendoCantidad:** Valida una lógica especial de inventario: en vez de eliminar completamente el registro, reduce la cantidad disponible del hardware cuando se realiza una operación de eliminación o descuento de stock.
- **contextLoads:** Verifica que Spring Boot cargue correctamente el contexto de la aplicación sin errores.

ix. Evidencias de resultados BUILD SUCCESS

BUILD SUCCESS

Tests run: X, Failures: 0, Errors: 0, Skipped: 0

→ Test Service-orden:

```

[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.56 s -- in com.pcer2.service_orden
.ServiceOrdenApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 23.420 s
[INFO] Finished at: 2026-06-20T21:14:12-04:00
[INFO] -----

```

→ Test Service-software:

```

the property 'springdoc.swagger-ui.enabled=false'
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 13.25 s -- in com.pcer2.service_softw
are.ServiceSoftwareApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 21.512 s
[INFO] Finished at: 2026-06-20T21:55:06-04:00
[INFO] -----

```

→ Test service-auth:

```

e property 'springdoc.swagger-ui.enabled=false'
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.946 s -- in com.pcer2.service_auth.
ServiceAuthApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.034 s
[INFO] Finished at: 2026-06-20T22:05:36-04:00
[INFO] -----

```


Conclusión

Durante la Experiencia 3, el proyecto **PCer2** evolucionó desde una definición inicial de requisitos hacia una solución backend más completa basada en microservicios. Se implementaron y actualizaron microservicios para clientes, equipos, servicios, órdenes, vouchers, hardware, software, mantenimiento, estadísticas, descuentos y autenticación.

Además, se incorporó API Gateway como punto de entrada centralizado, autenticación por token mediante service-auth, protección de rutas mediante AuthenticationFilter, Swagger/OpenAPI como herramienta de documentación técnica y JUnit 5 con Mockito para validar la lógica de los servicios.

La autenticación por token mejora la seguridad del sistema, mientras que la comunicación entre microservicios mediante WebClient permite relacionar información entre módulos sin duplicar datos innecesariamente. Finalmente, el uso de GitHub, Postman, Swagger y pruebas unitarias contribuye a mejorar la organización, trazabilidad, documentación y calidad técnica del proyecto **PCer2**.
